KPIT | APEX LAB

File Operations in System Programming

Low-Level POSIX File Handling in Linux

Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

Why File Operations Matter?

- Everything in Linux is a file
  - Regular files
  - Directories
  - Devices (/dev/ttyS0, /dev/i2c-1)
  - Sockets
  - Pipes
- System programming starts with file operations
- Low-level, unbuffered, deterministic I/O
- Sensors, UART, GPIO, sysfs entries, I2C/SPI, flash devices — all exposed as files.

File Descriptor

- All system calls performing IO refer to open files using a file descriptor
- a **nonnegative** number
- All file operations are performed via a file descriptor
- Kernel maintains a Per-process list of files → File Table
- FD table is per-process and stored in PCB

Process → FD Table → Open File Table → Inode

File Descriptor Number	File Descriptor Name	Description
0	STDIN_FILENO	Standard Input (Terminal Input Device)
1	STDOUT_FILENO	Standard Output (Terminal Display)
2	STDERR_FILENO	Standard Error (Terminal Error)

Core System Calls

Operation	System Call	Description
Open file	open()	Returns file descriptor
Read file	read()	Reads raw bytes
Write file	write()	Writes raw bytes
Move position	lseek()	Seek inside file
Control device	ioctl()	Device-specific operations
Close file	close()	Releases descriptor

- These are unbuffered, POSIX-standard calls.
- They provide predictable and deterministic I/O — perfect for embedded.

open()

- In order to perform any operation like read or write a file, need to open the file and provide handle to the kernel.
- FD values increase sequentially: next free FD is assigned.

File Descriptor	Description
0	Standard Input (Keyboard)
1	Standard Output (Screen)
2	Standard Error
3	May be user created file
.	

open()

- The **open**() system call opens the file specified by *path*.
- Headers Required

  #include <fcntl.h>

int open(const char ****path*, int** *flags***, int** *mode* **);**

int open(const char ****path*, int** *flags***);**

- path: file path
- Flags:
  - O_RDONLY, O_WRONLY, O_RDWR
  - O_CREAT (create if not exists; **requires** a mode like 0644)
  - O_TRUNC (truncate to 0 on open if writing)
  - O_APPEND (writes always append to end)
  - O_EXCL (with O_CREAT, fail if file exists)
- mode: file perms used only when O_CREAT
- Return: new file descriptor >=0. On error: -1

Sample Code (1/3)

Opening a file

Step 1: Develop a program

```
#include <fcntl.h>
#include <errno.h>
#include <stdio.h>

int main(void) {
int fd = open("demo.txt", O_RDONLY);
if (fd == -1) {
   printf("open() failed %d\n", errno);
   perror("ERROR");
return 1;
}
else{
   printf("The open() system call successful!\n");
   printf("The file Descriptor of open file is %d \n", fd);
}
return 0;
}
```

Sample Code (2/3)

Step 2: Compile and execute the program.

Output:

```
open() failed 2
ERROR: No such file or directory
```

Step 3: Create a file

```
$ demo.txt touch demo.txt
```

Step 4: Compile and execute

```
The open() system call successful!
The file Descriptor of open file is 3
```

Sample Code (3/3)

Step 5: Check file permissions

```
rw-r--r--
```

Step 6: Change permission as mentioned

```
---r--r--
```

Step 7: Compile and execute

```
open() failed 13
ERROR: Permission denied
```

read()

- Used to read the data from file

- Headers Required
  #include <unistd.h>

```
ssize_t read(int fd, void *buffer, size_t count)
```

- fd: file descriptor of the file to be read
- Buffer – provides the address of memory buffer where data needs to be placed
- count: number of bytes to read

- Return number of bytes read, -1 on error

Sample Code – Read from file

```
char buf[BUF_SIZE];
ssize_t nread;
nread = read(fd, buf, 20);
if (nread == -1) {
    printf("read() failed %d\n", errno);
    perror("ERROR");
    return 1;
}
else{
    buf[nread] = '\0';
    printf("The number of bytes read is %ld and read data is \"%s\" \n", nread, buf);
}
```

write()

- Used to write the data to the opened file

- #include <unistd.h>

```
ssize_t write(int fd, void *buffer, size_t count)
```

- Return number of bytes written
- Buffer – provides the address of memory buffer where data needs to be placed

Sample Code – Write to file

```
const char *msg = "Hello from demoNew.txt!\nThis line was written by write().\n";
ssize_t to_write = (ssize_t)strlen(msg);
ssize_t written = write(fd, msg, (size_t)to_write);
if (written == -1) {
    printf("write() failed %d\n", errno);
    perror("ERROR");
    return 1;
}
else{
    printf("The number of bytes written is %ld \n", written);
}
```

lseek()

- A file can be considered as a continuous set of bytes
- lssek is used to change the location of read/write pointer of a file descriptor

```
off_t lseek(int fd, off_t offset, int whence)
```

- offset: displacement.
- whence: SEEK_SET, SEEK_CUR, SEEK_END.

Feature	Meaning
SEEK_SET	Move relative to beginning
SEEK_CUR	Move relative to current position
SEEK_END	Move relative to EOF
Returns	New offset
Errors	ESPIPE, EINVAL, EBADF
Works with	Regular files, block devices
Not supported	Pipes / Sockets / FIFOs

```
lseek(fd, 0, SEEK_SET); // rewind to start
```

close()

- Used to close a file.
- close() reduces refcount of open file description.
- After close(), FD can be reused by open().

```
int close(int fd)
```

- fd: file descriptor to close.

```
if (close(fd) == -1){
   printf("close() failed %d\n", errno);
}
else{
   printf("The file is closed successfully!\n");
}
```

Robust File I/O — Error Handling

- Check:
  - Return values (-1 indicates error)
  - errno for specific failures
  - Handle partial reads/writes (common on sockets & pipes)
  - Recover gracefully on EAGAIN, EINTR